

rgbCTF 2020 Crypto - N-AES

deuterium — 2020-07-13

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/rgbctf/crypto/N-AES/2020-07-13-rgbCTF-2020-N-AES.html>

Abstract

CTF writeup: rgbCTF 2020's N-AES has more layers than entropy: brute-force a fixed one-byte seed to undo 128 AES layers, minding Python's int-versus-bytes seed trap.

N-AES

Description

```
What if I encrypt something with AES multiple times? nc challenge.rgbsec.xyz 34567

~qpwoeirut#5057
```

Files

- [n_aes.py](#)

```
import binascii
from base64 import b64encode, b64decode
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from os import urandom
from random import seed, randint

BLOCK_SIZE = 16

def rand_block(key_seed=urandom(1)):
    seed(key_seed)
    return bytes([randint(0, 255) for _ in range(BLOCK_SIZE)])

def encrypt(plaintext, seed_bytes):
    ciphertext = pad(b64decode(plaintext), BLOCK_SIZE)
    seed_bytes = b64decode(seed_bytes)
    assert len(seed_bytes) >= 8
    for seed in seed_bytes:
        ciphertext = AES.new(rand_block(seed), AES.MODE_ECB).encrypt(ciphertext)

    return b64encode(ciphertext)

def decrypt(ciphertext, seed_bytes):
    plaintext = b64decode(ciphertext)
    seed_bytes = b64decode(seed_bytes)
```

```

    for byte in reversed(seed_bytes):
        plaintext = AES.new(rand_block(byte), AES.MODE_ECB).decrypt(plaintext)

    return b64encode(unpad(plaintext, BLOCK_SIZE))

def gen_chall(text):
    text = pad(text, BLOCK_SIZE)
    for i in range(128):
        text = AES.new(rand_block(), AES.MODE_ECB).encrypt(text)

    return b64encode(text)

def main():
    challenge = b64encode(urandom(64))
    print(gen_chall(challenge).decode())
    while True:
        print("[1] Encrypt")
        print("[2] Decrypt")
        print("[3] Solve challenge")
        print("[4] Give up")

        command = input("> ")

        try:
            if command == '1':
                text = input("Enter text to encrypt, in base64: ")
                seed_bytes = input("Enter key, in base64: ")
                print(encrypt(text, seed_bytes))
            elif command == '2':
                text = input("Enter text to decrypt, in base64: ")
                seed_bytes = input("Enter key, in base64: ")
                print(decrypt(text, seed_bytes))
            elif command == '3':
                answer = input("Enter the decrypted challenge, in base64: ")
                if b64decode(answer) == challenge:
                    print("Correct!")
                    print("Here's your flag:")
                    with open("flag", 'r') as f:
                        print(f.read())
                else:
                    print("Incorrect!")
                    break
            elif command == '4':
                break
            else:
                print("Invalid command!")
        except binascii.Error:
            print("Base64 error!")
        except Exception:
            print("Error!")

    print("Bye!")

if __name__ == '__main__':
    main()

```

On netcatting, we get a base64 encoded encryption of a base64 encoded random string of 64 bytes.

```
challenge = b64encode(urandom(64))
print(gen_chall(challenge).decode())
```

Taking a look at gen_chall

```
def gen_chall(text):
    text = pad(text, BLOCK_SIZE)
    for i in range(128):
        text = AES.new(rand_block(), AES.MODE_ECB).encrypt(text)

    return b64encode(text)
```

And

```
def rand_block(key_seed=urandom(1)):
    seed(key_seed)
    return bytes([randint(0, 255) for _ in range(BLOCK_SIZE)])
```

This seems quite tricky, since rand_block will be presenting some random key and gen_chall is encrypting with some random key 128 times! right?

WRONG, There are some few caveats which we might exploit ;)

- Since no key_seed is specified in the gen_chall call to rand_block, it should be taking key_seed to be urandom(1) which is simply one byte :)
- More importantly, once it gets called, key_seed is fixed! So all the random blocks would essentially be the same! One may test it out.

```
from os import urandom
from random import seed, randint

BLOCK_SIZE = 16

def rand_block(key_seed=urandom(1)):
    seed(key_seed)
    return bytes([randint(0, 255) for _ in range(BLOCK_SIZE)])

for i in range(10):
    print(rand_block())

#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xcf5\x81o*\xce\x97\x90^90\x96T9~w'
```

```
#b'h\xf5\x81o*\xce\x97\x90^90\x96T9~w'
#b'h\xf5\x81o*\xce\x97\x90^90\x96T9~w'
```

So all that needs to be done is find out that random byte with which seed was initialised, and we will know the key, just decrypt our way out of the flag.

Solution

```
from pwn import remote
from base64 import b64encode, b64decode
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from random import seed, randint
import re
HOST, PORT = "challenge.rgbsec.xyz", 34567
REM = remote(HOST, PORT)

CHALL = b64decode(REM.recvline().strip())
def rand_block(byte):
    """random block for given seed byte"""
    seed(byte)
    return bytes([randint(0,255) for _ in range(16) ])

REM.recvuntil(b'\n>')

def dec_serv(ciphertext, seed_bytes):
    """Requests decryption from the server"""
    REM.sendline(b'2')
    REM.sendline(b64encode(ciphertext))
    REM.sendline(b64encode(seed_bytes))
    data = REM.recvuntil(b'\n>')
    if b'Error' not in data:
        decd = re.search(b'b\[([a-zA-Z0-9\+/\+])\]',data)[1]
        return b64decode(decd)

for i in range(256):
    decryption = dec_serv(CHALL, bytes([i]*128))
    if decryption:
        print(decryption)
        break

REM.sendline(b'3')
REM.sendline(b64encode(decrypt(CHALL)))
print(REM.recvregex(b'rgbCTF{.*}').decode())
```

WAIT! THAT WONT WORK!!

Tbh, I expected that to work but it didnt! Why?

Because server uses this decryption routine

```
def decrypt(ciphertext, seed_bytes):
    plaintext = b64decode(ciphertext)
    seed_bytes = b64decode(seed_bytes)
    for byte in reversed(seed_bytes):
        plaintext = AES.new(rand_block(byte), AES.MODE_ECB).decrypt(plaintext)
```

```
return b64encode(unpad(plaintext, BLOCK_SIZE))
```

Still cant spot it out?

All the devil is in `rand_block(byte)`. How? Because when byte objects are iterated upon, all the individual bytes are returned as int.

```
for i in b'a':
    print(i,type(i))

#97 <class 'int'>
```

Hmm, very interesting. But how does that make a difference?

Because `rand_block(i)` and `rand_block(byte([i]))` are completely different for an int `i`! Why? Because internally `seed(key_seed)` is used to initialize, and `seed(byte([i]))` and `seed(i)` are different! WTF!!

This implies the server would ~~not~~ never be able to decrypt using its own decryption routine!

To fix this, all we need to do is to write our own!

We know the decryption is correct just by looking at correct padding, since $\text{len}(\text{b64encode}(64 \text{ random bytes})) = 64 * 4 / 3 = 85$ and we have a ciphertext of len 96.

1 in AES initialization suggests `AES.MODE_ECB`

```
def decrypt(ct):
    ct_orig = ct
    for i in range(256):
        ct = ct_orig
        for _ in range(128):
            ct = AES.new(rand_block(bytes([i])),1).decrypt(ct)
        try:
            return unpad(ct,16)
        except:
            continue
```

Putting the final script in [solve.py](#).

```
from pwn import remote
from base64 import b64encode, b64decode
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from random import seed, randint
import re
HOST, PORT = "challenge.rgbsec.xyz", 34567
REM = remote(HOST, PORT)

CHALL = b64decode(REM.recvline().strip())
def rand_block(byte):
    """random block for given seed"""
    seed(byte)
```

```
        return bytes([randint(0,255) for _ in range(16) ])

REM.recvuntil(b'\n>')

def decrypt(ct):
    ct_orig = ct
    for i in range(256):
        ct = ct_orig
        for _ in range(128):
            ct = AES.new(rand_block(bytes([i])),1).decrypt(ct)
        try:
            return unpad(ct,16)
        except:
            continue

REM.sendline(b'3')
REM.sendline(b64encode(decrypt(CHALL)))
print(REM.recvregex(b'rgbCTF{.*}').decode())
```

rgbCTF-i'd0nt'7hink'7his'd03s'wh47'y0u'7hink'i7'd03s"

Interactive controls are omitted from this PDF. See the canonical HTML version.